



INSTITUTO TECNOLÓGICO DE MORELIA

INTELIGENCIA ARTIFICIAL

REPORTE DE PRACTICA U4

PRESENTA:

Gabriel Omar Valencia Sanchez

ASESOR:

ALCARAZ CHAVEZ JESUS EDUARDO

INDICE

INTRODUCCION	3
PLANTEAMIENTO DEL PROBLEMA	4
JUSTIFICACIÓN	5
OBJETIVOS	6
Objetivo general.....	6
Objetivos específicos	6
METODOLOGÍA.....	7
Análisis del problema y definición de requisitos.....	7
Diseño del conjunto de datos.....	8
Proceso de entrenamiento del tutor	9
Definición de reglas de comportamiento del tutor	10
DESARROLLO DEL SISTEMA	11
Estructura general del sistema	11
Flujo de funcionamiento del tutor	11
Interacción con el usuario	12
Control del contenido generado	12
Herramientas, modelo utilizado y dependencias del sistema	12
EVALUACIÓN Y ANÁLISIS DE RESULTADOS.....	14
Análisis del comportamiento del tutor	14
CONCLUSION	15

INTRODUCCION

En la actualidad, el uso de herramientas digitales para apoyar el aprendizaje se ha convertido en una práctica común dentro del ámbito educativo, especialmente en carreras relacionadas con la ingeniería y la tecnología. Los estudiantes cuentan cada vez con más recursos en línea que les permiten resolver dudas, practicar ejercicios y reforzar conceptos vistos en clase. Sin embargo, no todas estas herramientas ofrecen información confiable o explicaciones adecuadas para el nivel académico requerido.

Uno de los avances más recientes en este campo es el desarrollo de sistemas basados en inteligencia artificial capaces de generar texto de manera automática. Estos sistemas pueden responder preguntas, explicar conceptos y simular una conversación con un tutor humano. A pesar de su potencial, presentan un problema importante: en muchos casos generan respuestas incorrectas o información que no ha sido verificada, lo cual puede causar confusión en el estudiante.

En asignaturas como algoritmos y programación, este problema se vuelve aún más relevante. Los algoritmos son la base de la solución de problemas en ingeniería, y comprenderlos de manera incorrecta puede afectar el desarrollo académico del estudiante en cursos posteriores. Por esta razón, resulta necesario contar con herramientas que no solo expliquen, sino que lo hagan de forma clara, controlada y alineada con los contenidos académicos.

La presente práctica se centra en el desarrollo de un Tutor Inteligente de Algoritmos, diseñado específicamente para apoyar el aprendizaje de estudiantes de ingeniería. Este tutor fue entrenado utilizando información previamente seleccionada y validada, con el objetivo de asegurar que sus respuestas sean coherentes, comprensibles y correctas. Además, el sistema fue diseñado para no responder cuando no cuenta con información suficiente, evitando así la generación de errores.

PLANTEAMIENTO DEL PROBLEMA

El principal problema que se aborda en esta práctica es la falta de control en las respuestas generadas por sistemas de inteligencia artificial de uso general. Aunque estas herramientas son capaces de producir explicaciones aparentemente correctas, no siempre garantizan que la información proporcionada sea precisa o adecuada para un contexto educativo.

En el aprendizaje de algoritmos, los estudiantes suelen enfrentar dificultades relacionadas con la comprensión de conceptos abstractos, como el funcionamiento de ciclos, estructuras de decisión, análisis de eficiencia o estructuras de datos. Cuando buscan apoyo en herramientas automáticas, existe el riesgo de recibir explicaciones incompletas, confusas o incluso incorrectas, lo que puede reforzar errores conceptuales.

Otro problema identificado es que muchas herramientas ofrecen respuestas demasiado técnicas o, por el contrario, excesivamente simplificadas, lo que dificulta que el estudiante encuentre un equilibrio adecuado para su nivel académico. Además, la mayoría de estos sistemas no distinguen entre distintos tipos de respuestas, como explicaciones teóricas, guías paso a paso o retroalimentación conceptual.

Ante esta situación, surge la necesidad de desarrollar un sistema que actúe como un tutor académico, capaz de adaptar su forma de responder al contexto educativo y al tipo de pregunta realizada. Este tutor debe ser confiable, comprensible y estar limitado a un conjunto de conocimientos previamente definidos.

JUSTIFICACIÓN

El desarrollo de un Tutor Inteligente de Algoritmos se justifica por la necesidad de mejorar el apoyo académico en áreas fundamentales de la ingeniería. Los algoritmos son una de las bases del pensamiento lógico y computacional, y su correcta comprensión es esencial para el desempeño profesional de un ingeniero en sistemas.

Contar con una herramienta que brinde explicaciones claras y controladas permite a los estudiantes reforzar su aprendizaje fuera del aula, practicar de manera autónoma y aclarar dudas sin depender exclusivamente del tiempo limitado del docente. Además, un tutor inteligente puede ofrecer una experiencia de aprendizaje más personalizada, adaptándose al tipo de pregunta o dificultad presentada.

Desde el punto de vista académico, este proyecto demuestra la aplicación práctica de conceptos relacionados con inteligencia artificial y sistemas inteligentes, integrándolos en un contexto educativo real. Asimismo, permite explorar los límites y responsabilidades del uso de estas tecnologías, enfatizando la importancia del control y la validación de la información.

Finalmente, este trabajo sienta las bases para futuras investigaciones y desarrollos relacionados con el uso de inteligencia artificial en la educación, mostrando que es posible crear sistemas útiles y confiables cuando se diseñan con un enfoque adecuado.

OBJETIVOS

Objetivo general

Desarrollar un tutor inteligente enfocado en el aprendizaje de algoritmos y programación, capaz de brindar explicaciones claras, confiables y alineadas con contenidos académicos, utilizando un sistema de inteligencia artificial entrenado con información previamente seleccionada y controlada.

Este objetivo busca que el tutor no funcione únicamente como una herramienta de consulta rápida, sino como un apoyo educativo que ayude al estudiante a comprender los conceptos fundamentales de los algoritmos, reforzando el aprendizaje visto en clase y fomentando el razonamiento lógico.

Objetivos específicos

- Diseñar un conjunto de datos enfocado en temas de algoritmos, estructuras de datos y programación básica, utilizando explicaciones con enfoque educativo.
- Limpiar y organizar el conjunto de datos para asegurar que las respuestas sean coherentes, comprensibles y adecuadas al nivel de ingeniería.
- Entrenar un sistema de inteligencia artificial para que adopte un comportamiento similar al de un tutor académico.
- Implementar mecanismos que eviten que el tutor genere respuestas cuando no exista información suficiente.
- Evaluar el desempeño del tutor mediante el análisis de la calidad, claridad y coherencia de las respuestas generadas.
- Analizar las ventajas y limitaciones del uso de un tutor inteligente como apoyo en el aprendizaje de algoritmos.

METODOLOGÍA

La metodología seguida en esta práctica se diseñó con el objetivo de desarrollar un tutor inteligente confiable y adecuado para el contexto académico de ingeniería. Para ello, se establecieron una serie de etapas ordenadas que permitieron construir el sistema de manera progresiva, asegurando en cada paso la calidad de la información y el correcto funcionamiento del tutor.

El enfoque metodológico se basa en la idea de que un sistema educativo apoyado por inteligencia artificial debe ser desarrollado de forma estructurada, partiendo de una correcta definición del problema y concluyendo con una evaluación objetiva de los resultados obtenidos.

Análisis del problema y definición de requisitos

La primera etapa consistió en analizar las principales dificultades que enfrentan los estudiantes al aprender algoritmos. Se identificó que muchos estudiantes tienen problemas para comprender conceptos abstractos y para relacionar la teoría con la práctica. Además, se observó que las herramientas automáticas disponibles suelen ofrecer respuestas poco confiables o demasiado complejas.

A partir de este análisis, se definieron los siguientes requisitos principales para el tutor inteligente:

- Las respuestas deben ser claras y comprensibles para estudiantes de ingeniería.
- El sistema no debe generar información fuera del contenido previamente definido.
- El tutor debe priorizar la explicación conceptual sobre la entrega directa de soluciones.
- El sistema debe indicar explícitamente cuando no cuenta con información suficiente para responder.

Estos requisitos sirvieron como base para el diseño del sistema y guiaron todas las decisiones tomadas durante el desarrollo.

Diseño del conjunto de datos

```
dataset_final_v1_with_source.jsonl
dataset_final_v3_with_source.jsonl

{"id": "3c46524375", "mode": "bigtech", "instruction": "?Para qu? sirve el algoritmo de detecci?n de componentes fuertemente conexas (Kosaraju/Tarjan) paso a paso?", "input": "", "output": ""}
{"id": "2bb9f91f889", "mode": "general", "instruction": "?Qu? es un algoritmo?", "input": "", "output": "Imagina que vas a preparar un s?ndwich. Primero tomas el pan, luego agregas jam?n y queso. ?Qu? es un algoritmo? Es como preparar un s?ndwich: primero tomas el pan, luego agregas jam?n y queso."}
{"id": "12a53f938a", "mode": "general", "instruction": "Define qu? significa un algoritmo?", "input": "", "output": "Imagina que vas a preparar un s?ndwich. Primero tomas el pan, luego agregas jam?n y queso. ?Qu? es un algoritmo? Es como preparar un s?ndwich: primero tomas el pan, luego agregas jam?n y queso."}
{"id": "8678faa155", "mode": "general", "instruction": "Explicame el concepto de un algoritmo?", "input": "", "output": "Imagina que vas a preparar un s?ndwich. Primero tomas el pan, luego agregas jam?n y queso. ?Qu? es un algoritmo? Es como preparar un s?ndwich: primero tomas el pan, luego agregas jam?n y queso."}
{"id": "e52848d4b2", "mode": "general", "instruction": "?Qu? es un algoritmo con ejemplos de c?digo?", "input": "", "output": "Imagina que vas a preparar un s?ndwich. Primero tomas el pan, luego agregas jam?n y queso. ?Qu? es un algoritmo? Es como preparar un s?ndwich: primero tomas el pan, luego agregas jam?n y queso."}
{"id": "8c82ea8cdf", "mode": "general", "instruction": "?Qu? es un algoritmo paso a paso?", "input": "", "output": "Imagina que vas a preparar un s?ndwich. Primero tomas el pan, luego agregas jam?n y queso. ?Qu? es un algoritmo? Es como preparar un s?ndwich: primero tomas el pan, luego agregas jam?n y queso."}
{"id": "0bfc655844", "mode": "general", "instruction": "?Qu? es una variable?", "input": "", "output": "Piensa en una variable como una cajita con una etiqueta. Puedes ponerle dentro un n?mero, un texto o incluso otro objeto. ?Qu? es una variable? Es como una cajita con una etiqueta."}
{"id": "407a0a6e08", "mode": "general", "instruction": "Define qu? significa una variable?", "input": "", "output": "Piensa en una variable como una cajita con una etiqueta. Puedes ponerle dentro un n?mero, un texto o incluso otro objeto. ?Qu? es una variable? Es como una cajita con una etiqueta."}
{"id": "ac94828598", "mode": "general", "instruction": "Explicame el concepto de una variable?", "input": "", "output": "Piensa en una variable como una cajita con una etiqueta. Puedes ponerle dentro un n?mero, un texto o incluso otro objeto. ?Qu? es una variable? Es como una cajita con una etiqueta."}
{"id": "17262f5b30", "mode": "general", "instruction": "?Qu? es una variable con ejemplos de c?digo?", "input": "", "output": "Piensa en una variable como una cajita con una etiqueta. Puedes ponerle dentro un n?mero, un texto o incluso otro objeto. ?Qu? es una variable? Es como una cajita con una etiqueta."}
{"id": "e31e0ed7b7", "mode": "general", "instruction": "?Qu? es una variable paso a paso?", "input": "", "output": "Piensa en una variable como una cajita con una etiqueta. Puedes ponerle dentro un n?mero, un texto o incluso otro objeto. ?Qu? es una variable? Es como una cajita con una etiqueta."}
{"id": "af687c4203", "mode": "general", "instruction": "Explica el concepto de recursividad de forma sencilla", "input": "", "output": "La recursividad es como una mu?eca rusa cuando se abre. Cada mu?eca contiene una mu?eca m?s peque?a. ?Qu? es la recursividad? Es como una mu?eca rusa que se abre para revelar una mu?eca m?s peque?a."}
{"id": "668dc852c8", "mode": "general", "instruction": "Dame una explicaci?n sobre el concepto de recursividad de forma sencilla", "input": "", "output": "La recursividad es como una mu?eca rusa cuando se abre. Cada mu?eca contiene una mu?eca m?s peque?a. ?Qu? es la recursividad? Es como una mu?eca rusa que se abre para revelar una mu?eca m?s peque?a."}
{"id": "861607d705", "mode": "general", "instruction": "Ay?dame a entender el concepto de recursividad de forma sencilla", "input": "", "output": "La recursividad es como una mu?eca rusa cuando se abre. Cada mu?eca contiene una mu?eca m?s peque?a. ?Qu? es la recursividad? Es como una mu?eca rusa que se abre para revelar una mu?eca m?s peque?a."}
{"id": "e1ace35d90", "mode": "general", "instruction": "Explica el concepto de recursividad de forma sencilla con ejemplos de c?digo", "input": "", "output": "La recursividad es como una mu?eca rusa cuando se abre. Cada mu?eca contiene una mu?eca m?s peque?a. ?Qu? es la recursividad? Es como una mu?eca rusa que se abre para revelar una mu?eca m?s peque?a."}
{"id": "7253f55c5b", "mode": "general", "instruction": "Explica el concepto de recursividad de forma sencilla paso a paso", "input": "", "output": "La recursividad es como una mu?eca rusa cuando se abre. Cada mu?eca contiene una mu?eca m?s peque?a. ?Qu? es la recursividad? Es como una mu?eca rusa que se abre para revelar una mu?eca m?s peque?a."}
{"id": "7b34c990f7", "mode": "general", "instruction": "?Qu? es el factorial y c?mo se calcula con recursividad?", "input": "", "output": "Me puedes dar una explicaci?n f?cil?", "output": "Imagina que quieres calcular el factorial de 5. Puedes pensar en ello como 5 * 4 * 3 * 2 * 1. ?Qu? es el factorial? Es el producto de todos los n?meros positivos menores o iguales a un n?mero dado."}
{"id": "2e8dda1d67", "mode": "general", "instruction": "Define qu? significa el factorial y c?mo se calcula con recursividad?", "input": "", "output": "Me puedes dar una explicaci?n f?cil?", "output": "Imagina que quieres calcular el factorial de 5. Puedes pensar en ello como 5 * 4 * 3 * 2 * 1. ?Qu? es el factorial? Es el producto de todos los n?meros positivos menores o iguales a un n?mero dado."}
{"id": "e869f8e194", "mode": "general", "instruction": "Explicame el concepto de el factorial y c?mo se calcula con recursividad?", "input": "", "output": "Me puedes dar una explicaci?n f?cil?", "output": "Imagina que quieres calcular el factorial de 5. Puedes pensar en ello como 5 * 4 * 3 * 2 * 1. ?Qu? es el factorial? Es el producto de todos los n?meros positivos menores o iguales a un n?mero dado."}
{"id": "4397d0cc51", "mode": "general", "instruction": "?Qu? es el factorial y c?mo se calcula con recursividad con ejemplos de c?digo?", "input": "", "output": "Me puedes dar una explicaci?n f?cil?", "output": "Imagina que quieres calcular el factorial de 5. Puedes pensar en ello como 5 * 4 * 3 * 2 * 1. ?Qu? es el factorial? Es el producto de todos los n?meros positivos menores o iguales a un n?mero dado."}
{"id": "3c1025e425", "mode": "general", "instruction": "?Qu? es el factorial y c?mo se calcula con recursividad paso a paso?", "input": "", "output": "Me puedes dar una explicaci?n f?cil?", "output": "Imagina que quieres calcular el factorial de 5. Puedes pensar en ello como 5 * 4 * 3 * 2 * 1. ?Qu? es el factorial? Es el producto de todos los n?meros positivos menores o iguales a un n?mero dado."}
{"id": "719785f8e4", "mode": "general", "instruction": "?Qu? es Fibonacci con recursividad?", "input": "", "output": "La secuencia de Fibonacci es como criar conejos. Empiezas con una pareja y cada par de meses producen una nueva pareja. ?Qu? es Fibonacci? Es la secuencia de n?meros donde cada n?mero es la suma de los dos anteriores."}
{"id": "e2c2546b34", "mode": "general", "instruction": "Define qu? significa Fibonacci con recursividad?", "input": "", "output": "La secuencia de Fibonacci es como criar conejos. Empiezas con una pareja y cada par de meses producen una nueva pareja. ?Qu? es Fibonacci? Es la secuencia de n?meros donde cada n?mero es la suma de los dos anteriores."}
{"id": "93f4c4d377", "mode": "general", "instruction": "Explicame el concepto de Fibonacci con recursividad", "input": "", "output": "La secuencia de Fibonacci es como criar conejos. Empiezas con una pareja y cada par de meses producen una nueva pareja. ?Qu? es Fibonacci? Es la secuencia de n?meros donde cada n?mero es la suma de los dos anteriores."}
{"id": "9c33bf1a1e", "mode": "general", "instruction": "?Qu? es Fibonacci con recursividad con ejemplos de c?digo?", "input": "", "output": "La secuencia de Fibonacci es como criar conejos. Empiezas con una pareja y cada par de meses producen una nueva pareja. ?Qu? es Fibonacci? Es la secuencia de n?meros donde cada n?mero es la suma de los dos anteriores."}
{"id": "eeae3e4e7f", "mode": "general", "instruction": "?Qu? es Fibonacci con recursividad paso a paso?", "input": "", "output": "La secuencia de Fibonacci es como criar conejos. Empiezas con una pareja y cada par de meses producen una nueva pareja. ?Qu? es Fibonacci? Es la secuencia de n?meros donde cada n?mero es la suma de los dos anteriores."}
{"id": "9228f498b6", "mode": "bigtech", "instruction": "?Para qu? sirve un bucle en programaci?n?", "input": "", "output": "Un bucle es como cuando saltas la cuerda 10 veces. En vez de escribir el mismo c?digo 10 veces, puedes usar un bucle para hacerlo una sola vez. ?Qu? es un bucle? Es una estructura de control que repite un bloque de c?digo."}
{"id": "3e694e28ee", "mode": "general", "instruction": "?Cu?l es la utilidad de un bucle en programaci?n?", "input": "", "output": "Un bucle es como cuando saltas la cuerda 10 veces. En vez de escribir el mismo c?digo 10 veces, puedes usar un bucle para hacerlo una sola vez. ?Qu? es un bucle? Es una estructura de control que repite un bloque de c?digo."}
{"id": "43cf645a43", "mode": "general", "instruction": "?En qu? casos se usa un bucle en programaci?n?", "input": "", "output": "Un bucle es como cuando saltas la cuerda 10 veces. En vez de escribir el mismo c?digo 10 veces, puedes usar un bucle para hacerlo una sola vez. ?Qu? es un bucle? Es una estructura de control que repite un bloque de c?digo."}
{"id": "18d7d8ae6f", "mode": "bigtech", "instruction": "?Para qu? sirve un bucle en programaci?n con ejemplos de c?digo?", "input": "", "output": "Un bucle es como cuando saltas la cuerda 10 veces. En vez de escribir el mismo c?digo 10 veces, puedes usar un bucle para hacerlo una sola vez. ?Qu? es un bucle? Es una estructura de control que repite un bloque de c?digo."}
{"id": "729c7dbf9c", "mode": "bigtech", "instruction": "?Para qu? sirve un bucle en programaci?n paso a paso?", "input": "", "output": "Un bucle es como cuando saltas la cuerda 10 veces. En vez de escribir el mismo c?digo 10 veces, puedes usar un bucle para hacerlo una sola vez. ?Qu? es un bucle? Es una estructura de control que repite un bloque de c?digo."}
{"id": "728095f866", "mode": "general", "instruction": "?Qu? diferencia hay entre un bucle 'for' y uno 'while'?", "input": "", "output": "Piensa en un bucle for como cuando haces una lista de cosas que quieres comprar. Sabes exactamente cu?ntas cosas quieres. ?Qu? es la diferencia? El bucle 'for' se usa cuando sabes cu?ntas veces quieres repetir algo, y el bucle 'while' se usa cuando no sabes cu?ntas veces."}
{"id": "782b199c90", "mode": "general", "instruction": "?Qu? diferencia hay entre un bucle 'for' y uno 'while' con ejemplos de c?digo?", "input": "", "output": "Piensa en un bucle for como cuando haces una lista de cosas que quieres comprar. Sabes exactamente cu?ntas cosas quieres. ?Qu? es la diferencia? El bucle 'for' se usa cuando sabes cu?ntas veces quieres repetir algo, y el bucle 'while' se usa cuando no sabes cu?ntas veces."}
{"id": "74fad5ee85", "mode": "general", "instruction": "?Qu? diferencia hay entre un bucle 'for' y uno 'while' paso a paso?", "input": "", "output": "Piensa en un bucle for como cuando haces una lista de cosas que quieres comprar. Sabes exactamente cu?ntas cosas quieres. ?Qu? es la diferencia? El bucle 'for' se usa cuando sabes cu?ntas veces quieres repetir algo, y el bucle 'while' se usa cuando no sabes cu?ntas veces."}
{"id": "749cf88b82", "mode": "general", "instruction": "?Qu? es un array?", "input": "", "output": "Un array es como un tren. Cada vag?n (posici?n) puede llevar un objeto un n?mero, un texto o incluso otro objeto. ?Qu? es un array? Es una estructura de datos que almacena una colecci?n ordenada de elementos."}
{"id": "ff519e09f4", "mode": "general", "instruction": "Define qu? significa un array?", "input": "", "output": "Un array es como un tren. Cada vag?n (posici?n) puede llevar un objeto un n?mero, un texto o incluso otro objeto. ?Qu? es un array? Es una estructura de datos que almacena una colecci?n ordenada de elementos."}
{"id": "7f666abab1", "mode": "general", "instruction": "Explicame el concepto de un array?", "input": "", "output": "Un array es como un tren. Cada vag?n (posici?n) puede llevar un objeto un n?mero, un texto o incluso otro objeto. ?Qu? es un array? Es una estructura de datos que almacena una colecci?n ordenada de elementos."}
{"id": "8d45671778", "mode": "general", "instruction": "?Qu? es un array", "input": "", "output": "Un array es como un tren. Cada vag?n (posici?n) puede llevar un objeto un n?mero, un texto o incluso otro objeto. ?Qu? es un array? Es una estructura de datos que almacena una colecci?n ordenada de elementos."}
{"id": "762a5b8aaa", "mode": "general", "instruction": "?Qu? es un array", "input": "", "output": "Un array es como un tren. Cada vag?n (posici?n) puede llevar un objeto un n?mero, un texto o incluso otro objeto. ?Qu? es un array? Es una estructura de datos que almacena una colecci?n ordenada de elementos."}
{"id": "810a491b14", "mode": "general", "instruction": "?Qu? es un array", "input": "", "output": "Un array es como un tren. Cada vag?n (posici?n) puede llevar un objeto un n?mero, un texto o incluso otro objeto. ?Qu? es un array? Es una estructura de datos que almacena una colecci?n ordenada de elementos."}
```

Una vez definidos los requisitos, se procedi? al dise?o del conjunto de datos que ser? utilizado para entrenar al tutor. Este conjunto de datos se enfoc? exclusivamente en temas relacionados con algoritmos, programaci?n y estructuras de datos, buscando cubrir los contenidos b?sicos que normalmente se abordan en cursos de ingenier?a.

Cada registro del conjunto de datos incluye una pregunta o instrucci?n y una respuesta redactada con un enfoque educativo. Se evit? el uso excesivo de lenguaje t?cnico y se priorizaron explicaciones claras, ejemplos conceptuales y analog?as sencillas cuando fue necesario.

Adem?s, las respuestas fueron clasificadas seg?n su tipo, lo que permiti? que el tutor adoptara distintos comportamientos dependiendo del contexto de la pregunta. Esta clasificaci?n facilit? la creaci?n de un tutor m?s flexible y cercano al proceso de ense?anza real.

Proceso de entrenamiento del tutor

El entrenamiento del tutor se realizó utilizando un modelo de lenguaje previamente entrenado, el cual fue adaptado para el dominio específico de algoritmos. Este proceso permitió que el modelo aprovechara su conocimiento general del lenguaje, pero ajustando su comportamiento al contexto académico.

Durante el entrenamiento, se reforzó el estilo educativo de las respuestas, priorizando la claridad y la explicación paso a paso. También se establecieron reglas para evitar que el tutor generara respuestas completas cuando estas no eran pedagógicamente adecuadas.

El proceso de entrenamiento fue optimizado para utilizar de manera eficiente los recursos computacionales disponibles, lo que permitió completar la práctica en un tiempo razonable sin comprometer la calidad del modelo resultante.

```
train_tutor_v2_fixed3py
C:\Unidad1> train_tutor_v2_fixed3py
1 import argparse
2 import os
3 import inspect
4
5 from datasets import load_dataset
6 from transformers import (
7     AutoTokenizer,
8     AutoModelForCausalLM,
9     Trainer,
10     TrainingArguments,
11     DataCollatorForSeq2Seq,
12 )
13 import torch
14 from peft import LoraConfig, get_peft_model
15
16
17 def parse_args():
18     p = argparse.ArgumentParser(
19         "Lola fine-tune Tutor de Algoritmos (Qwen2.5) - Rápido y eficiente (padding dinámico)"
20     )
21     p.add_argument("--dataset_path", type=str, default="dataset_final_v3_with_source.jsonl")
22     p.add_argument("--model_name", type=str, default="Qwen/Qwen2.5-1.5B-Instruct")
23     p.add_argument("--output_dir", type=str, default="/tutor_algoritmos_lora_v3")
24     p.add_argument("--num_epochs", type=int, default=3)
25     p.add_argument("--batch_size", type=int, default=4)
26     p.add_argument("--gradient_accum", type=int, default=2)
27     p.add_argument("--learning_rate", type=float, default=1e-4)
28     p.add_argument("--max_length", type=int, default=512)
29     p.add_argument("--eval_ratio", type=float, default=0.1)
30     return p.parse_args()
31
32
33 SYSTEM_BY_MODE = {
34     "general": (
35         "Eres un Tutor Inteligente de Algoritmos. ",
36         "Explicas de forma clara, paso a paso y con paciencia. ",
37         "Si el conocimiento proviene del temario, usa el prefijo 'Va, del temario:'. ",
38         "Si es conocimiento externo, usa el prefijo 'Data extra (no del dataset):'. ",
39     )
40 }
41
42
43 def build_prompt(mode, instruction, inp, source=None):
44     """Construye el prompt de entrenamiento.
45
46     Agregamos 'source' para que el modelo aprenda a distinguir:
47     - source='dataset' -> conocimiento del temario/dataset
48     - source='extra' -> conocimiento externo/general
49     """
50     mode = (mode or "general").strip().lower()
51     instruction = (instruction or "").strip()
52     inp = (inp or "").strip()
53     source = (source or "dataset").strip().lower()
54
55     system = SYSTEM_BY_MODE.get(mode, SYSTEM_BY_MODE["general"])
56
57     prompt = system + "\n\n"
58     prompt += "### Source:\n" + source + "\n\n"
59     prompt += "### Instruction:\n" + instruction + "\n\n"
60     prompt += "### Input:\n" + inp if inp else "" + "\n"
```

Definición de reglas de comportamiento del tutor

Un aspecto clave de la metodología fue la definición explícita de reglas que el tutor debía seguir al interactuar con el usuario. Estas reglas se establecieron con el fin de asegurar un comportamiento coherente y alineado con el enfoque educativo del proyecto.

Entre las principales reglas definidas se encuentran:

- El tutor debe responder únicamente con información contenida en el conjunto de datos.
- El tutor debe utilizar un lenguaje claro y adecuado al nivel de ingeniería.
- En caso de no encontrar información relacionada con la pregunta, el tutor debe indicar que no puede responder.
- El tutor debe evitar respuestas excesivamente largas o técnicas.

Estas reglas permitieron construir un sistema predecible y confiable, adecuado para su uso como apoyo académico.

```
● Tutor listo
Escribe 'salir' para terminar.

Tú: quien es carlos V

Tutor: No tengo esa información en el dataset.

Tú: que es un algoritmo

Tutor: Va, del temario: Imagina que vas a preparar un sándwich. Primero tomas el pan, luego agregas jamón, después queso, y finalmente cierras con otro pan. ¡Eso es un algoritmo! Una serie de pasos ordenados para lograr un objetivo. En programación, un algoritmo es simplemente la receta para resolver un problema.

Tú: []
```

DESARROLLO DEL SISTEMA

El desarrollo del sistema se realizó integrando distintos módulos que trabajan de manera conjunta para ofrecer una experiencia de tutoría controlada y eficiente. Cada módulo cumple una función específica dentro del sistema y contribuye al funcionamiento general del tutor.

Estructura general del sistema

El sistema se compone de tres módulos principales: el módulo de entrenamiento, el módulo de evaluación y el módulo de interacción con el usuario. Esta separación permitió desarrollar y probar cada componente de manera independiente, facilitando el mantenimiento y la mejora del sistema.

El módulo de entrenamiento se encarga de adaptar el modelo al dominio de algoritmos, el módulo de evaluación analiza la calidad de las respuestas generadas y el módulo de interacción permite al usuario comunicarse con el tutor de manera directa.

Flujo de funcionamiento del tutor

El funcionamiento del tutor inicia cuando el usuario introduce una pregunta o consulta. El sistema analiza esta entrada y busca información relacionada dentro del conjunto de datos. Si se encuentra una coincidencia clara, el tutor devuelve la respuesta correspondiente, manteniendo el estilo educativo definido.

En caso de que no exista una coincidencia suficiente, el tutor informa al usuario que no cuenta con información relacionada. Este comportamiento evita la generación de respuestas incorrectas y refuerza la confiabilidad del sistema.

Este flujo de funcionamiento permite que el tutor actúe como una herramienta de apoyo académico, brindando respuestas solo cuando estas están respaldadas por información validada.

Interacción con el usuario

La interacción con el usuario fue diseñada para ser simple y directa. El estudiante puede realizar preguntas de forma natural, sin necesidad de utilizar comandos especiales o formatos complejos. Esto facilita el uso del tutor y reduce la barrera de entrada para estudiantes con distintos niveles de experiencia.

Además, el tutor mantiene un estilo de respuesta consistente, lo que ayuda al estudiante a familiarizarse rápidamente con la herramienta y a confiar en la información proporcionada.

Control del contenido generado

Uno de los aspectos más importantes del desarrollo del sistema es el control del contenido generado. Para lograrlo, se implementaron mecanismos que comparan la pregunta del usuario con el contenido del conjunto de datos. Solo cuando existe una coincidencia clara, el tutor genera una respuesta.

Este enfoque garantiza que el sistema no genere información fuera de su conocimiento definido y permite mantener un alto nivel de confiabilidad en las respuestas ofrecidas.

Herramientas, modelo utilizado y dependencias del sistema

El desarrollo del Tutor Inteligente de Algoritmos se realizó utilizando el lenguaje de programación Python, debido a su facilidad de uso y a la amplia disponibilidad de bibliotecas orientadas al desarrollo de sistemas educativos y de inteligencia artificial. Para evitar conflictos entre librerías, el proyecto se trabajó dentro de un entorno virtual de Python.

El modelo de lenguaje empleado fue Qwen2.5-1.5B-Instruct, un modelo preentrenado diseñado para responder instrucciones y explicar información de manera clara. Este modelo fue adaptado al dominio de algoritmos mediante un entrenamiento especializado, lo que permitió que el tutor adoptara un comportamiento académico y controlado.

Las dependencias necesarias para el funcionamiento del sistema incluyen bibliotecas encargadas del manejo del modelo de lenguaje, el procesamiento de datos y la evaluación de texto. Estas dependencias se encuentran definidas en el archivo requirements.txt y se instalan de manera sencilla utilizando el gestor de paquetes de Python.

La información utilizada por el tutor, así como los datos de evaluación, se almacenaron en archivos con formato JSONL, lo que facilitó la organización y el procesamiento de los datos. Para ejecutar correctamente el sistema se recomienda contar con Python versión 3.9 o superior, un sistema operativo compatible y al menos 8 GB de memoria RAM. El entrenamiento del modelo puede beneficiarse del uso de una tarjeta gráfica, aunque para su uso en modo consulta es suficiente contar con CPU.

EVALUACIÓN Y ANÁLISIS DE RESULTADOS

La evaluación del tutor inteligente se realizó con el objetivo de verificar si el sistema cumplía con los requisitos definidos desde el inicio de la práctica. En particular, se buscó comprobar que el tutor fuera capaz de ofrecer respuestas claras, coherentes y alineadas con el contenido académico, sin generar información incorrecta o fuera del conjunto de datos establecido.

Para llevar a cabo esta evaluación, se utilizó un conjunto de preguntas diferente al empleado durante el entrenamiento. De esta manera, fue posible analizar el comportamiento del tutor frente a situaciones nuevas y verificar si mantenía un desempeño consistente.

Análisis del comportamiento del tutor

El análisis del comportamiento del tutor permitió identificar varias fortalezas del sistema. Una de las principales es su capacidad para mantener un estilo de respuesta acorde al nivel de ingeniería, sin recurrir a explicaciones excesivamente técnicas ni demasiado simplificadas.

Asimismo, el tutor demuestra un comportamiento predecible y confiable, lo cual es fundamental en el ámbito educativo. El hecho de que el sistema indique cuando no puede responder refuerza la confianza del estudiante y evita la propagación de información incorrecta.

No obstante, también se identificaron algunas limitaciones. El tutor depende completamente del contenido del conjunto de datos, por lo que su cobertura temática está limitada. Esto implica que, para ampliar su utilidad, es necesario incrementar la cantidad y variedad de información disponible.

CONCLUSION

El desarrollo del Tutor Inteligente de Algoritmos permitió comprobar que es posible utilizar sistemas de inteligencia artificial como apoyo al aprendizaje, siempre que se diseñen con un enfoque adecuado y responsable. A lo largo de la práctica, se logró construir un sistema capaz de ofrecer explicaciones claras y controladas, alineadas con contenidos académicos de ingeniería.

Uno de los principales logros del proyecto es el control del conocimiento del tutor. Al limitar las respuestas a un conjunto de datos previamente definido, se redujo significativamente el riesgo de generar información incorrecta. Este aspecto resulta especialmente importante en asignaturas fundamentales como algoritmos, donde la precisión conceptual es esencial.

Además, el tutor demostró un comportamiento pedagógico adecuado, priorizando la explicación y el razonamiento sobre la entrega directa de respuestas. Esto contribuye al desarrollo del pensamiento lógico del estudiante y fomenta un aprendizaje más profundo.